

Week Six

More on Techniques and System Prompts

Prompt Engineering

Mick McQuaid

mcq@utexas.edu

University of Texas at Austin

18 Feb 2025

Week Six

Agenda

- Presentations: Kylie, Aditya
- News
- Review whatialreadyknow (Ishwari)
- eC review
- m1 review
- Haystack (again)
- Techniques
- System prompts
- Work time

Presentations

News

The Batch

⟨ pause to look at this week's edition ⟩

WhatIAlreadyKnow (Ishwari)

eC review

⟨ look at the doc ⟩

m1 review

- It's a colossal blunder to fail to turn in either the qmd or html file
- I'm going to suggest that almost everyone use Huggingface datasets instead of Kaggle
 - → For example, Huggingface has 24 Shakespeare datasets
- Each group should probably focus on one task, such as recommendation (3), question answering (1), or summarization (2) (One group has several tasks)
- Although you are using different platforms, you will be required to extensively document what you're doing in a qmd file and an html file for the three remaining milestones
- If you're using Jupyter, leave enough time to translate your notebook to quarto

Haystack

Break into *new* pairs as specified on Canvas.

Tutorial components

The tutorial shows you how to create a generative question-answering pipeline using the retrieval-augmentation (RAG) approach with Haystack 2.0. The process involves four main components:

- SentenceTransformersTextEmbedder for creating an embedding for the user query,
- InMemoryBM25Retriever for fetching relevant documents,
- PromptBuilder for creating a template prompt, and
- OpenAIChatGenerator for generating responses.

But first, data!

- quickStart
- tutorialData
- Try the Python fragments from the tutorial on the seve-wonders dataset
- If that's successful, try it on your own dataset
- The fragments are *Check dataset validity*, *List configuration and splits*, *Preview a dataset*, *Download slices of a dataset*, *Search text in a dataset*, *Access Parquet files*, and *Get the size of the dataset*.

Embed documents

- Once you have the data, you can embed it using the `SentenceTransformersTextEmbedder`. This will create an embedding for each document in the dataset.
- You could use a different embedder. Let's look at the list of embedders at <https://docs.haystack.deepset.ai/docs/embedders>. What you do will depend on the data you choose for your project.
- For now, let's use the default.
- By the way, what is an embedding?
- In its simplest form, an embedding is a vector of numbers that represents (in this case) a sentence. The vector is created by a neural network that has been trained to learn the meaning of the word or sentence.

Start building the RAG pipeline

- What is RAG, again?
- RAG is a retrieval-augmented generation approach. It uses a retriever to find relevant documents, and then uses a generator to generate a response.
- The pipeline generates a response by combining the retrieved documents with the user query.
- You will use a text embedder for the user's question that matches the document embedder you used to embed the documents.
- You will use a retriever to find relevant documents.
- You will define a prompt template that will be used to generate the response.
- You will use a chat generator to generate the response.

Initialize the pipeline

- The tutorial has initialization steps for the above components.
- The steps are:
 - Initialize the text embedder
 - Initialize the retriever
 - Initialize the prompt builder
 - Initialize the chat generator

Build the pipeline

- The tutorial has a build step for the above components.

Asking a question

- The tutorial has a run step for the above components.
- In this case, a single question is asked.
- The question is: “What does the Rhodes Statue look like?”
- Some examples are given that could be run in a loop.
- Your simple interface to the pipeline is probably going to be a simple loop.
- I’m not going to require any error-handling or user-friendly interface components. You can consider me the only user and I can be given instructions to follow, such as to press Ctrl-D to exit the loop.
- The interface need not be a web-based interface but can be. You are welcome to ask a model to generate a web-based interface but a simple command-line interface is sufficient.

Other Haystack tutorials

- Although you are not required to use Haystack, you might find it useful. There are many tutorials on the Haystack website, for example:
- **Filtering Documents with Metadata**
- **Preprocessing Different File Types**
- **Creating a Hybrid Retrieval Pipeline**

These are Beginner tutorials. There are also Intermediate and Advanced tutorials linked from the same page as the one we just did.

More on Techniques

Last time, we did few-shot and zero-shot prompting techniques. Now we'll move on to thought generation techniques and others. Again, we're referring to Schulhoff et al. ([2024](#)).

Thought Generation

- *prompting the model to articulate its ongoing reasoning*
- Chain-of-Thought
- Zero-Shot Chain-of-Thought
- Step-Back Prompting
- Analogical Prompting
- Thread-of-Thought Prompting
- Tabular Chain-of-Thought

Few-Shot CoT

- *multiple examples, including chains-of-thought*
- Contrastive CoT Prompting
- Uncertainty-Routed CoT Prompting
- Complexity-based Prompting
- Active Prompting
- Memory-of-Thought Prompting
- Automatic Chain-of-Thought Prompting

Decomposition

- *explicitly decomposing the problem into subproblems*
- Least-to-Most Prompting
- Plan-and-Solve Prompting
- Tree-of-Thought Prompting
- Recursion-of-Thought Prompting
- Program-of-Thoughts
- Faithful Chain-of-Thought
- Skeleton-of-Thought
- Metacognitive Prompting

Ensembling

- *using multiple prompts to solve the same problem, then aggregating the results, for example, by majority vote*
- Demonstration Ensembling
- Mixture of Reasoning Experts
- Max Mutual Information Method
- Self-Consistency
- Universal Self-Consistency
- Meta-Reasoning over Multiple CoTs
- DiVeRSe
- Consistency-based Self-Adaptive Prompting
- Universal Self-Adaptive Prompting
- Prompt Paraphrasing

Self-Criticism

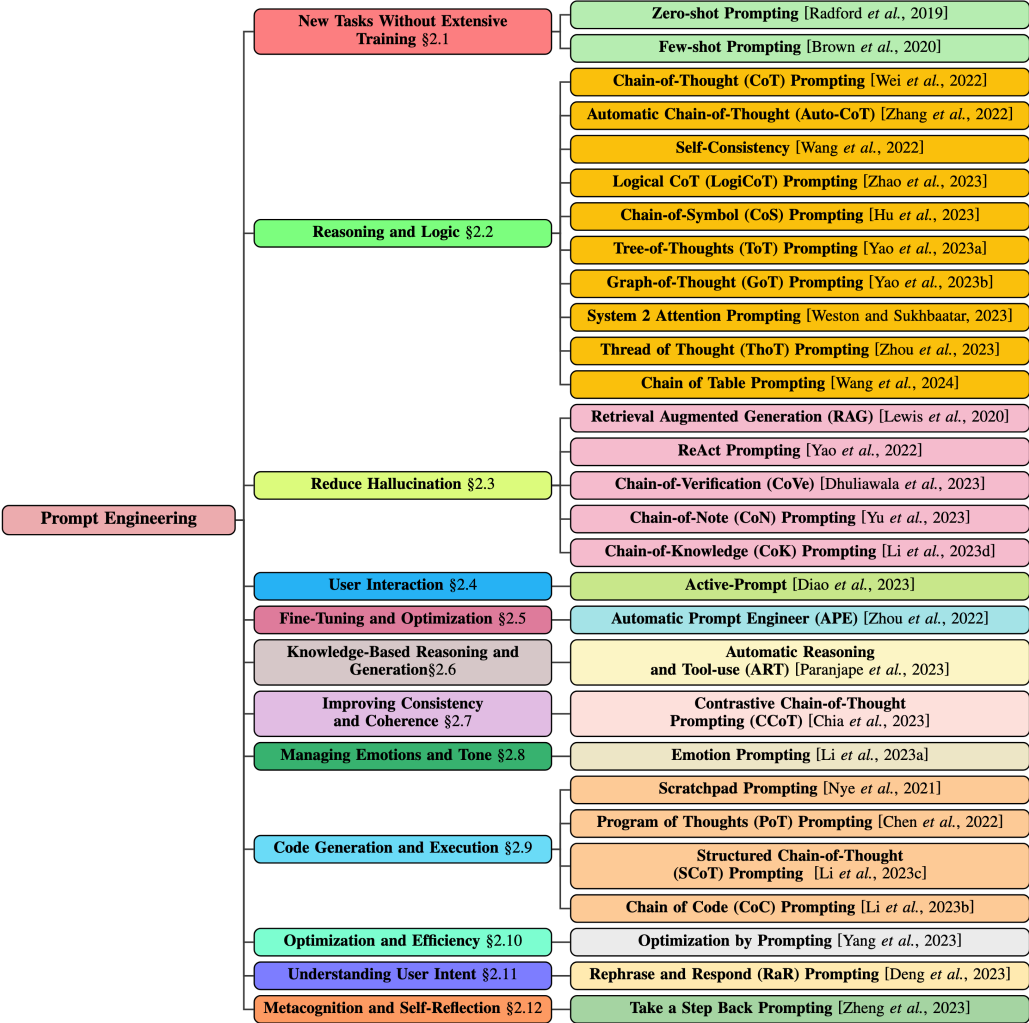
- *prompting the model to critique its own output*
- Self-Calibration
- Self-Refine
- Reversing Chain-of-Thought
- Self-Verification
- Chain-of-Verification
- Cumulative Reasoning

Prompt Engineering Techniques and Goals

Prompt Engineering Techniques

- Meta Prompting
- AutoPrompt
- Automatic Prompt Engineer
- Gradientfree Instructional Prompt Search
- Prompt Optimization with Textual Gradients
- RLPrompt
- Dialogue-comprised Policy-gradient-based Discreet Prompt Optimization

Tree of Prompt Engineering Techniques from Sahoo et al. (2024)



Goals of Prompt Engineering, Sahoo et al. (2024), 1 of 2

- New tasks without extensive training
- Reasoning and logic
- Reduce hallucination
- User interaction
- Fine-tuning and optimization
- Knowledge-based reasoning and generation
- Improving consistency and coherence

Goals of Prompt Engineering, Sahoo et al. (2024), 2 of 2

- Managing emotions and tone
- Code generation and execution
- Optimization and Efficiency
- Understanding User Intent
- Metacognition and Self-Reflection

Automatic Prompt Optimization, Wan et al. (2024)

- Prompt engineering can be automated through APO
- Two approaches:
 - → Instruction Optimization
 - → Exemplar Optimization
- They are synergistic, but
- → EO is easier and more effective

System Prompts, Zhang et al. (2024)

What system prompts are

- System prompts are general instructions that are not task specific but are included in a prompt
- Some authors call them meta-instructions
- An example might be “Let’s think step by step”
- Recent studies show that the state-of-the-art prompt optimizer is ProTeGi
- Accordingly, Zhang et al. (2024) test a system prompt optimizer with it and find that the system prompt optimizer performs better, noting that system prompts are more generally applicable

Zhang et al. (2024), continued

- Zhang et al. (2024) cite recent research showing that combining multiple benchmarks in a single evaluation improves efficiency and alignment with human preferences, so they evaluate on a lot of task types
- Zhang et al. (2024) uses a genetic algorithm to iterate over system prompts (tens of thousands at each step), relying on removing mistakes from output at each iteration

Example of successful system prompt

Write an answer that makes the reader feel happy. Write like you are explaining. First establish the set of facts you know, then answer the question based only on those facts.

Ethical considerations

Admirably, Zhang et al. (2024) includes a section on ethical considerations. These include:

- energy consumption of the process
- possibility of stereotyping people, e.g., “act like a professor”
- crowdsourced annotations from WEIRD contexts
- need for research on model performance in new cultural and social contexts

END

References

- Sahoo, Pranab, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2024. "A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications." <https://arxiv.org/abs/2402.07927>.
- Schulhoff, Sander, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, et al. 2024. "The Prompt Report: A Systematic Survey of Prompting Techniques." <https://arxiv.org/abs/2406.06608>.
- Wan, Xingchen, Ruoxi Sun, Hootan Nakhost, and Sercan O. Arik. 2024. "Teach Better or Show Smarter? On Instructions and Exemplars in Automatic Prompt Optimization." <https://arxiv.org/abs/2406.15708>.
- Zhang, Lechen, Tolga Ergen, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. 2024. "SPRIG: Improving Large Language Model Performance by System Prompt Optimization." <https://arxiv.org/abs/2410.14826>.

Colophon

This slideshow was produced using [quarto](#)

Fonts are *Roboto Light*, *Roboto Bold*, and *JetBrains Mono Nerd Font*